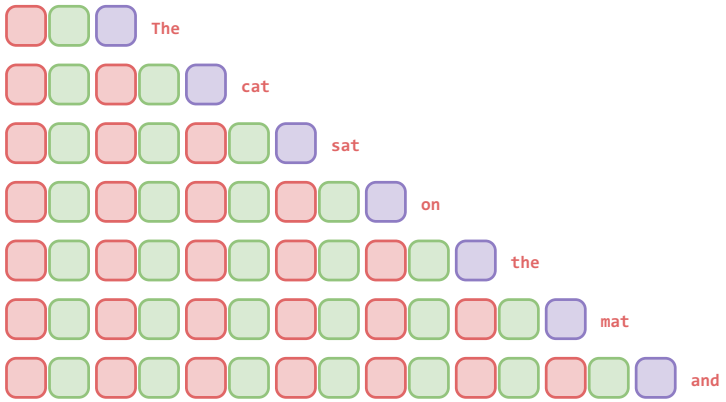
UNIT 01 · CHAPTER 10

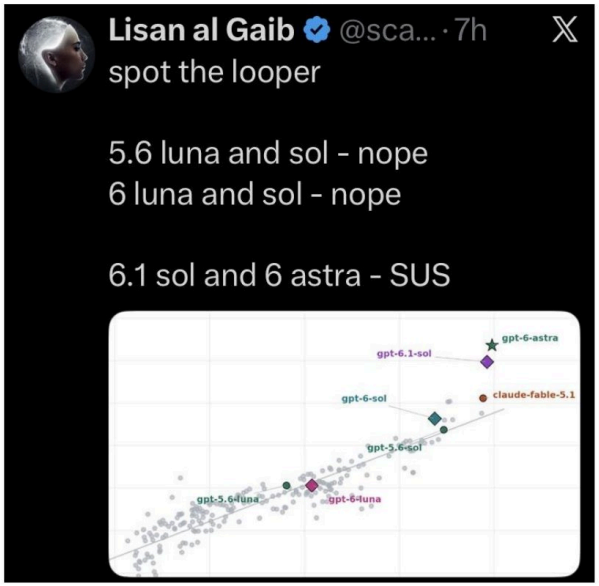
Attention Variants

CSE 10124 · Building ChatGPT

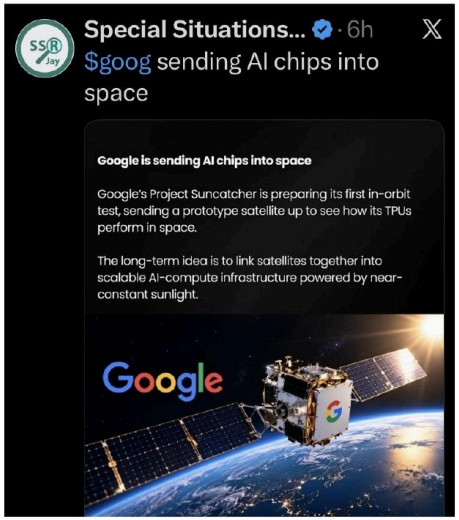
William Theisen · University of Notre Dame



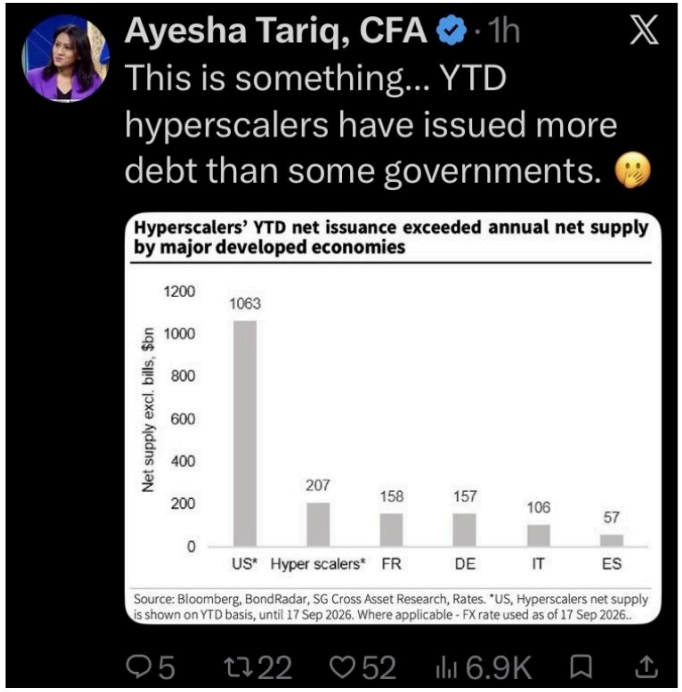
In the News


Lisan al Gaib @sca... · 7h
spot the looper
5.6 luna and sol - nope
6 luna and sol - nope
6.1 sol and 6 astra - SUS
gpt-6-astra
gpt-6.1-sol
claude-fable-5.1
gpt-6-sol
gpt-5.6-sol
gpt-5.6-luna
gpt-6-luna


etn. @etnshow · 4h
JUST IN: China puts travel restrictions on top AI founders and their families, who will now require approval from Beijing even for short trips, per Bloomberg.
16
109
576
54K

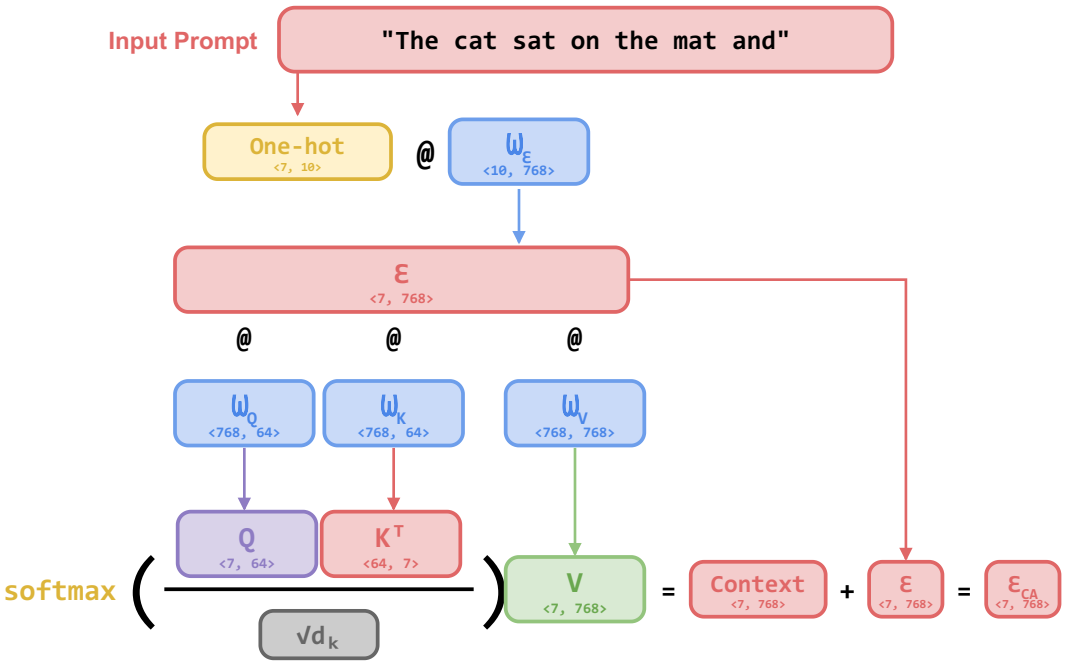

Special Situations... · 6h
$goog sending AI chips into space
Google is sending AI chips into space
Google's Project Suncatcher is preparing its first in-orbit test, sending a prototype satellite up to see how its TPUs perform in space.
The long-term idea is to link satellites together into scalable AI-compute infrastructure powered by near-constant sunlight.
Google

In the News

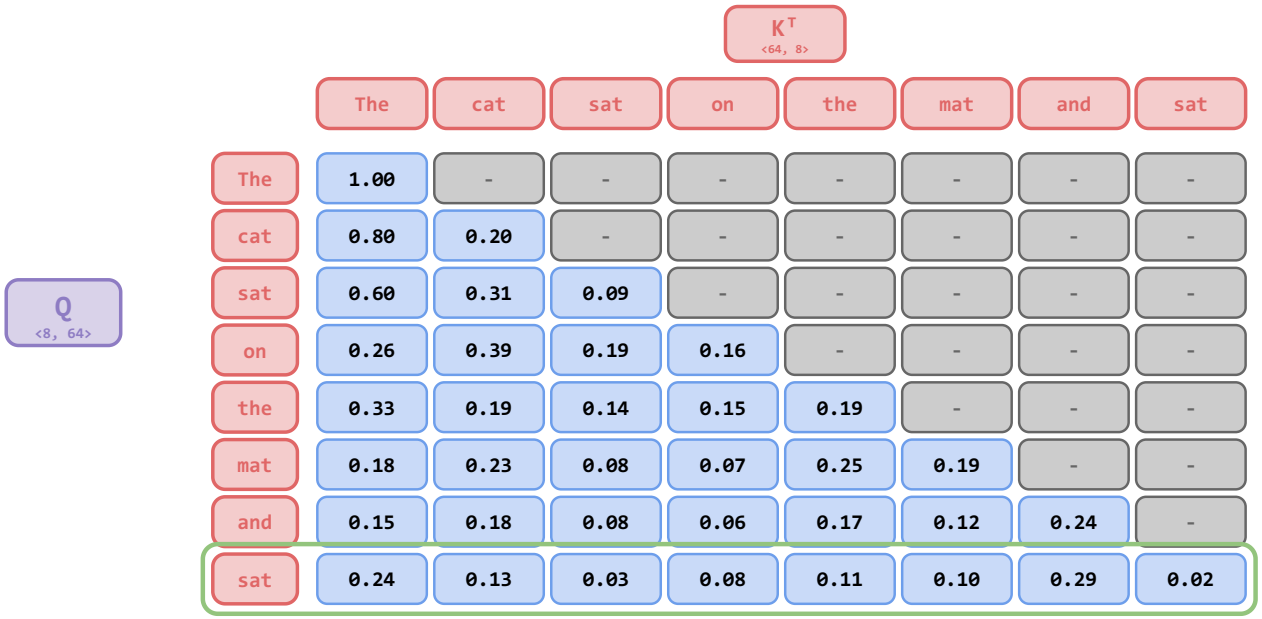

Ayesha Tariq, CFA · 1h
This is something... YTD hyperscalers have issued more debt than some governments.
Hyperscalers' YTD net issuance exceeded annual net supply by major developed economies
Net supply excl. bills, $bn
1200
1000
800
600
400
200
0
1063
207
158
157
106
57
US*
Hyper scalers*
FR
DE
IT
ES
Source: Bloomberg, BondRadar, SG Cross Asset Research, Rates. *US, Hyperscalers net supply is shown on YTD basis, until 17 Sep 2026. Where applicable - FX rate used as of 17 Sep 2026..
5
22
52
6.9K


bubble boi @bu... · 1m
Oh nooooooooooo Dario is pulling a Leopold on a much larger scale 💀💀💀
Ed Zitron @edzitron · 3h
Per Reuters, Anthropic has $252 billion in non-cancelable compute obligations across Microsoft, Google and Amazon, and $1...
billion with Alphabet's (GOOGL.O)
long-term infrastructure service obli
ion of Broadcom (AVGO.O) -related
ic filed for an IPO confidentially with t
720

Standard Attention



Auto-Regressive Text Generation



We can append the predicted token to our sequence, re-calculate attention, and then use that to predict what the next token should be, etc. etc.

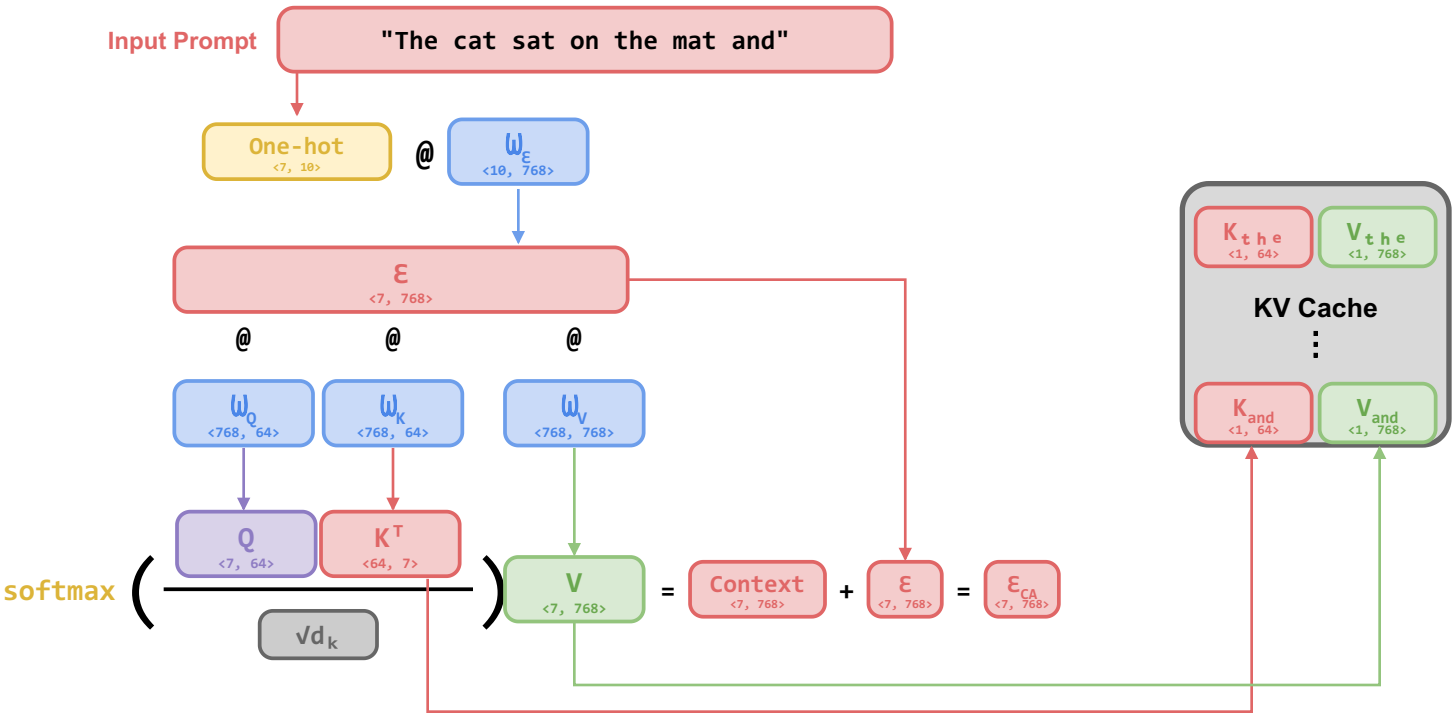
LLM Inference with a KV Cache

We can make **LLM inference** more efficient by using a **KV Cache**

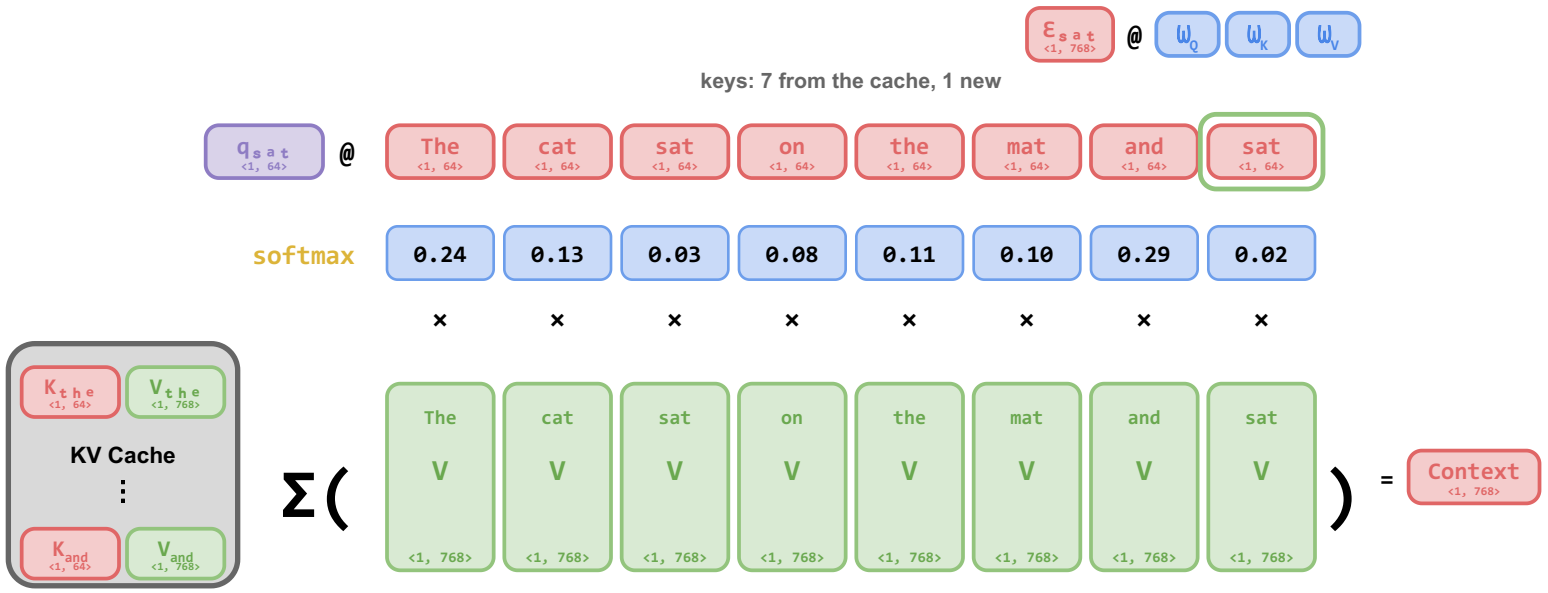
KV Cache: a "dictionary" that stores all of our previously computed **K**s and **V**s so we can re-use them (instead of needing to recalculate them) with future tokens

In **multi-head attention**, each head has its own **KV Cache**

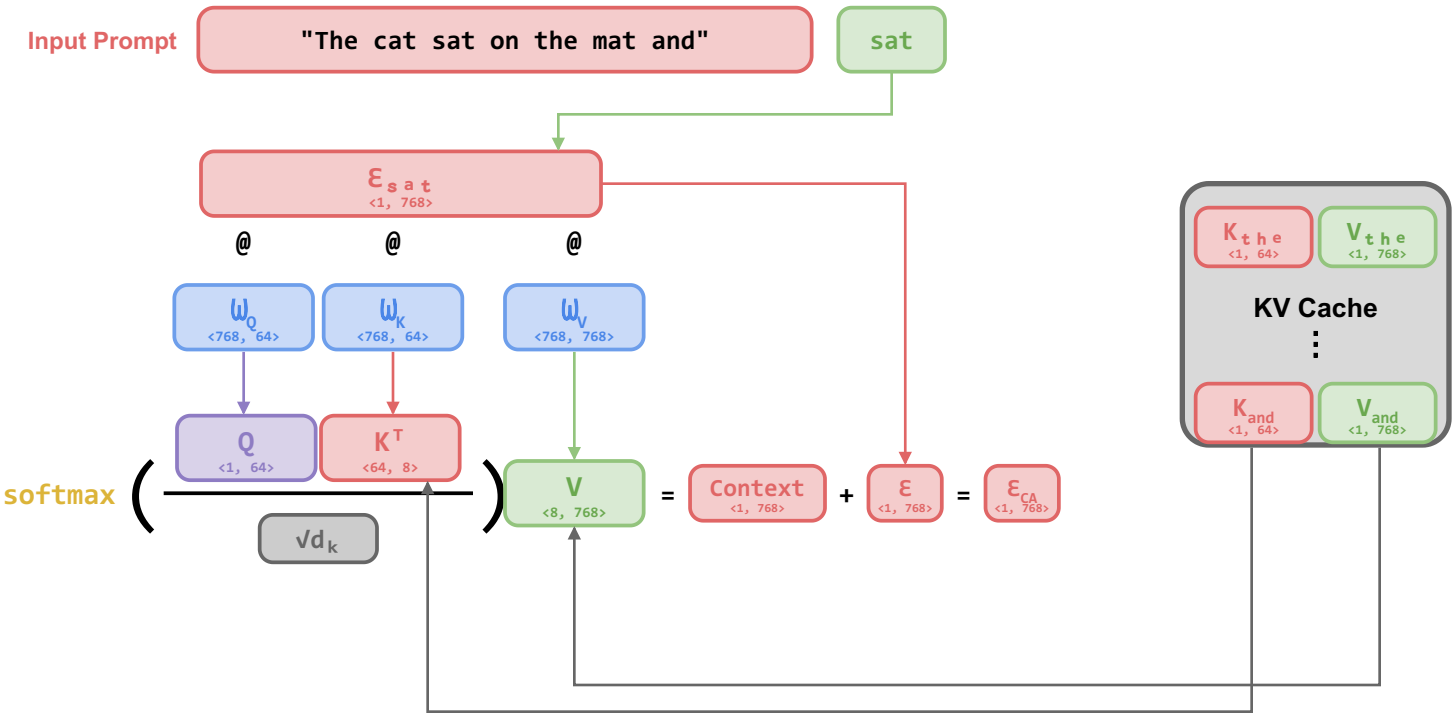
Standard Attention with KV Cache



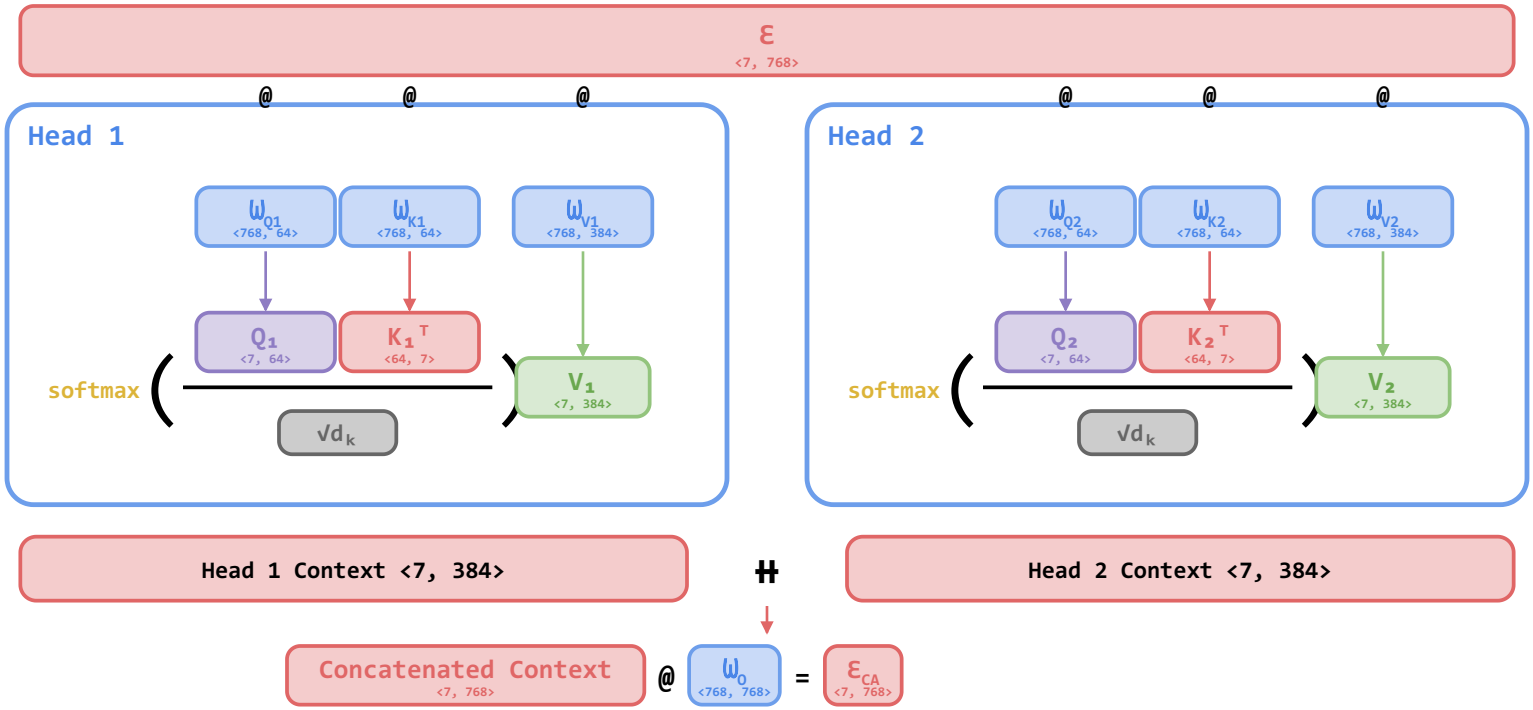
Calculating Attention with a KV Cache



Text Generation with a KV Cache



Variant 1: Multi-Head Attention



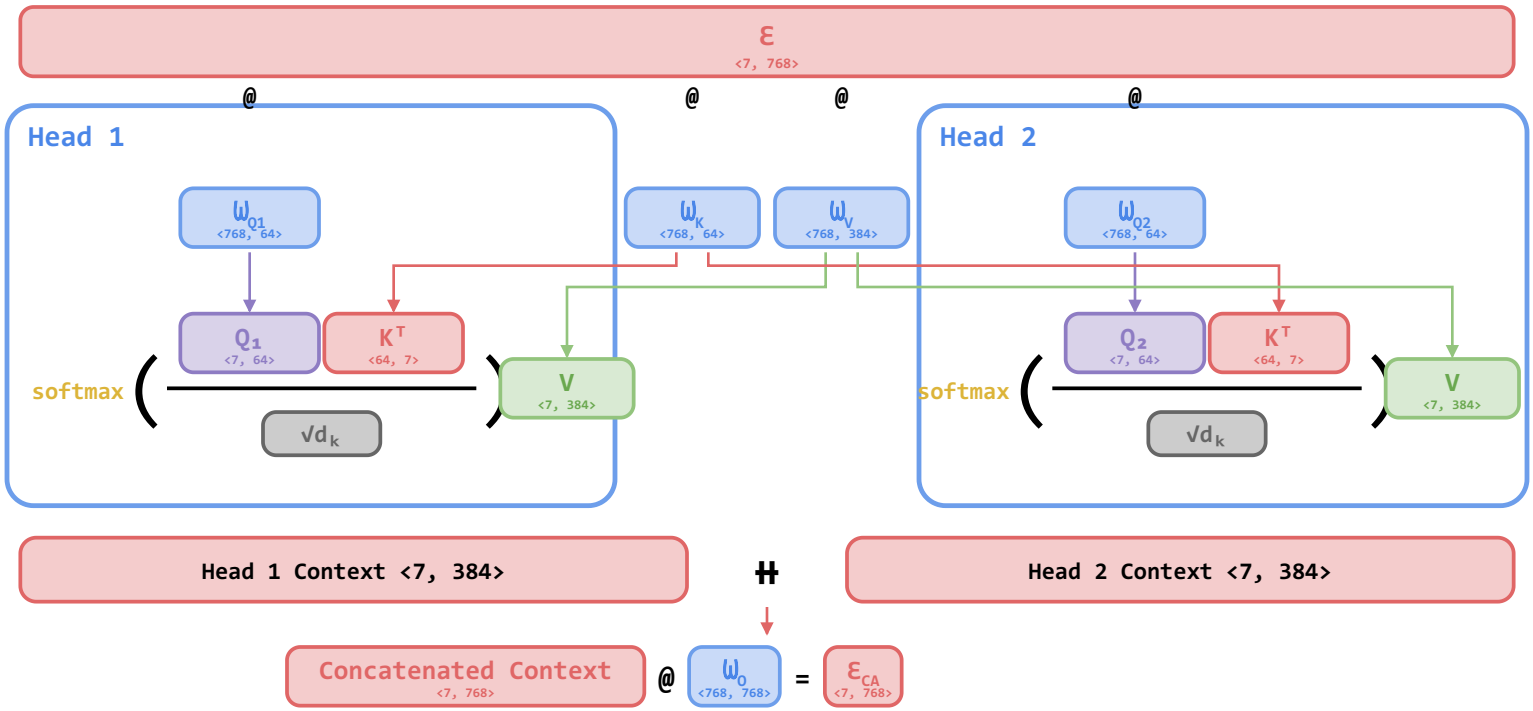
Attention in Today's LLMs



From Sebastian Raschka's LLM Architecture Gallery (sebastianraschka.com/llm-architecture-gallery)

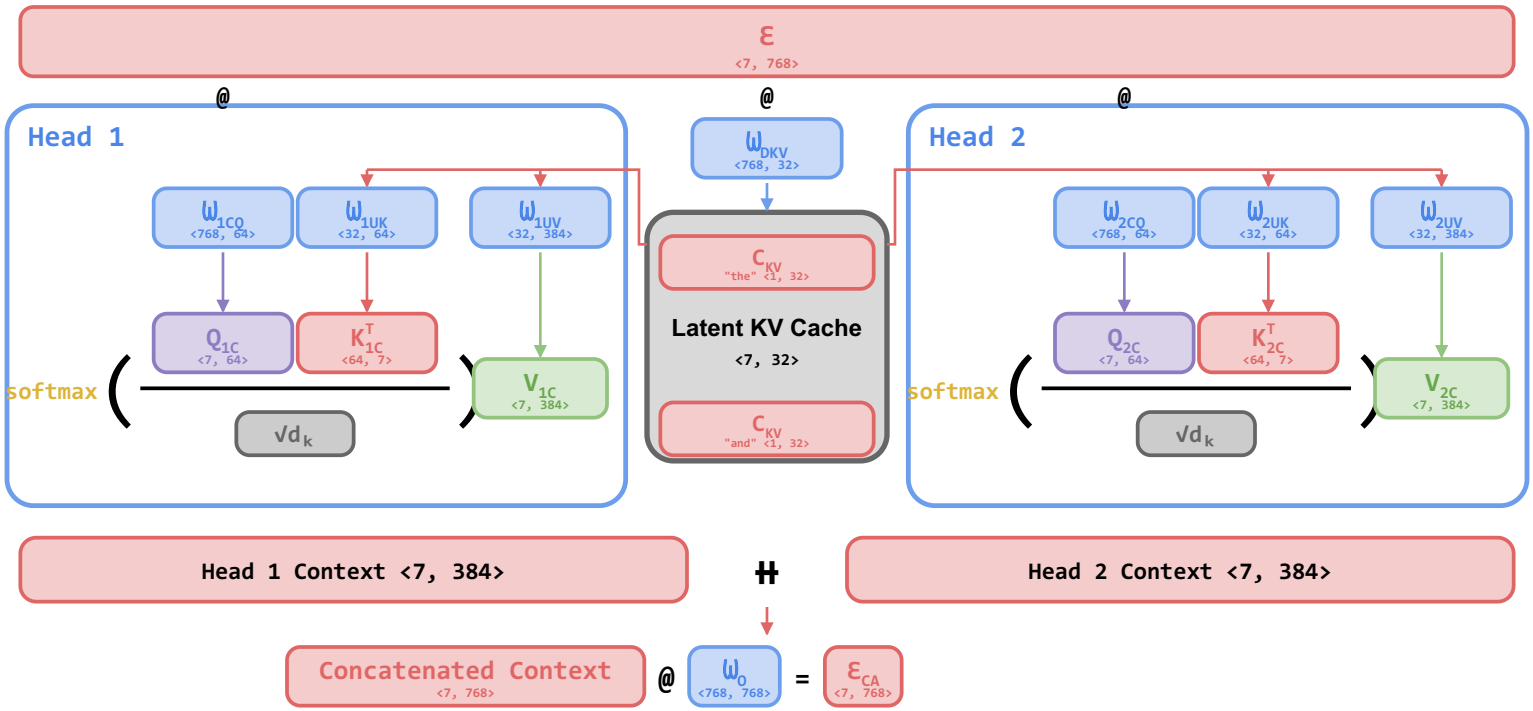
Variant 2: Grouped-Query Attention (GQA)

Llama 3 8B, Qwen3 4B, Gemma 3 27B, Mistral Small 3.1 24B, SmolLM3 3B



Variant 3-1: Multi-Head Latent Attention (MLA)

DeepSeek V3, Kimi K2, GLM-5, Ling 2.5, Mistral Large 3



LLM Inference: Prefill and Decode

LLM inference can be broken down into two separate steps: **Prefill** and **Decode**

Prefill: reading phase where the model **processes your entire input prompt** all at once in parallel (this fills the **KV cache**).

Decode: writing phase where the model **generates the output response** one token at a time, sequentially.

Decode with Multi-Head Latent Attention (MLA)

We can do some very clever math to make **MLA** inference more efficient by "absorbing" two of our matrices

Absorbing the key up-projection

$$Q_{iC} \; K_{iC}^{T} = (\varepsilon \; @ \; W_{iCQ}) \; @ \; (C_{KV} \; @ \; W_{iUK})^{T}$$

$$= \varepsilon \; @ \; W_{iCQ} \; @ \; W_{iUK}^{T} \; @ \; C_{KV}^{T} \qquad (AB)^{T} = B^{T}A^{T}$$

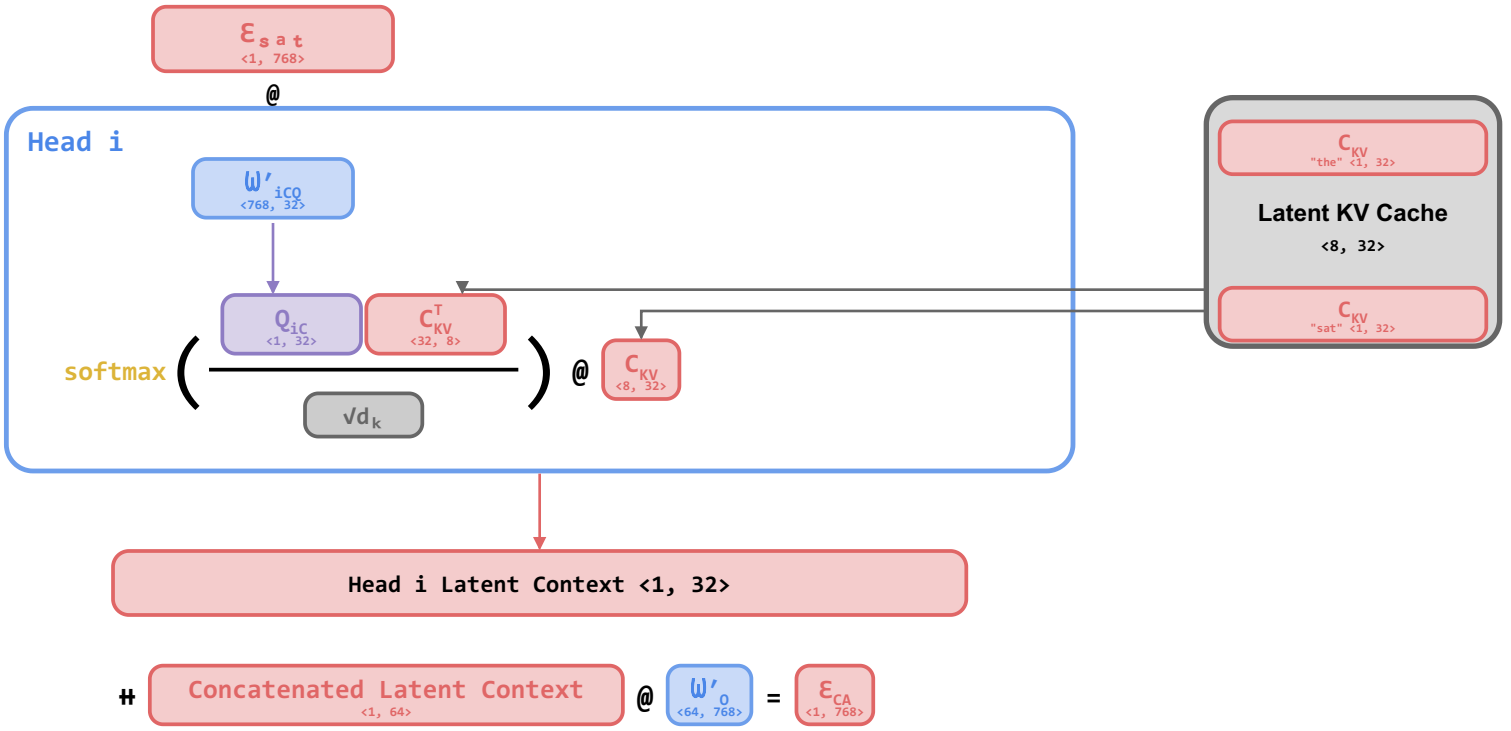
$$= \varepsilon \; @ \; W'_{iCQ} \; @ \; C_{KV}^{T} \qquad \text{the two blue tiles above, multiplied once}$$

Absorbing the value up-projection

$$\text{Context}_{i} \; @ \; W_{O,i} = \text{softmax}(\ldots) \; @ \; C_{KV} \; @ \; W_{iUV} \; @ \; W_{O,i}$$

$$= \text{softmax}(\ldots) \; @ \; C_{KV} \; @ \; W'_{O,i} \qquad \text{the last two tiles, multiplied once}$$

Variant 3-2: Inference with Multi-Head Latent Attention (MLA)



Variant 4: Sliding Window Attention (SWA)

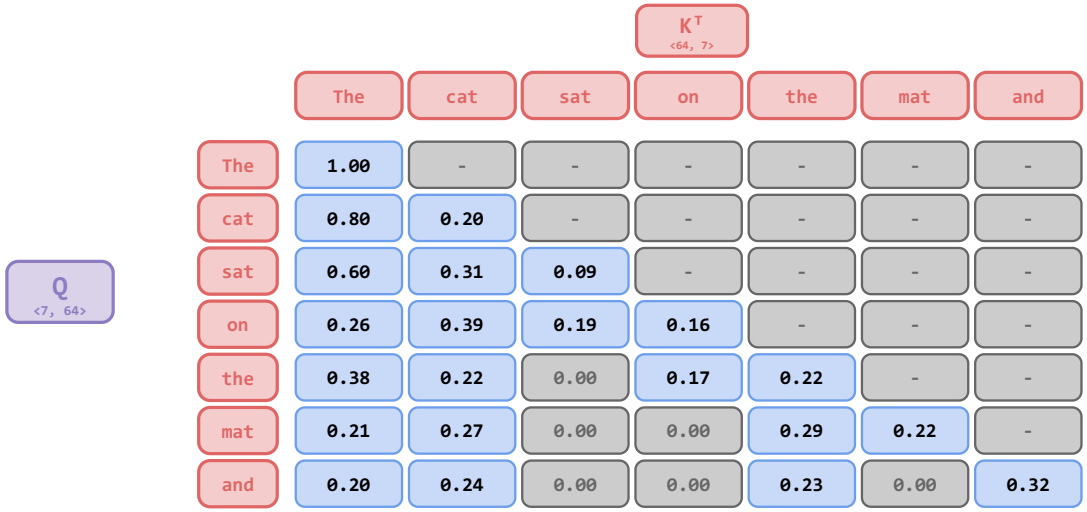
Gemma 3 27B, OLMo 3 32B, Xiaomi MiMo-V2-Flash, Arcee Trinity, Step 3.5 Flash



This would be `window_size=3`

Variant 5: DeepSeek Sparse Attention (DSA)

DeepSeek V3.2 (DSA + MLA) and GLM-5

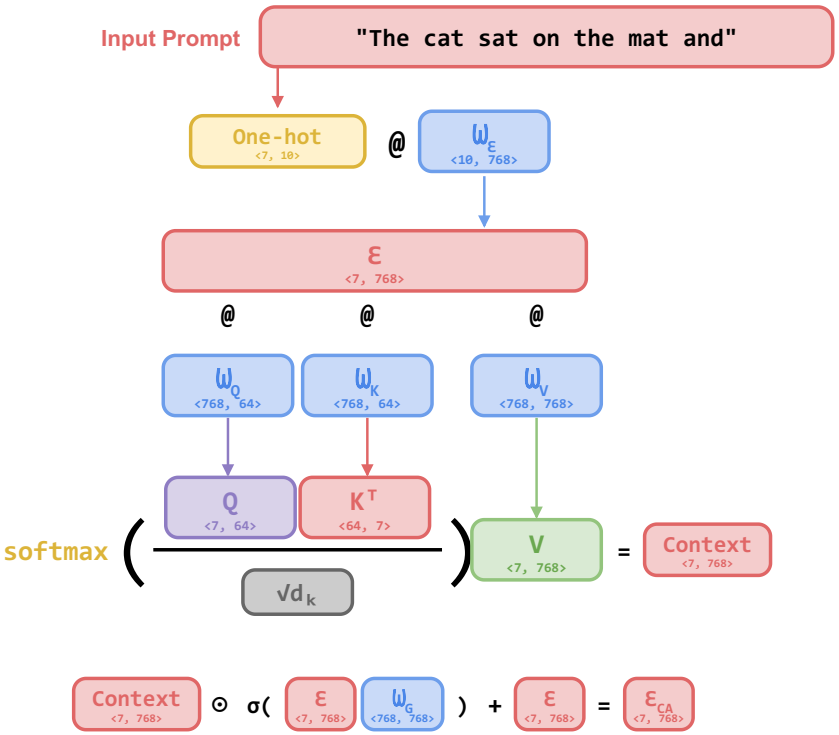


We calculate an importance score for each token and attend to only the k most important ones (this would be `top_k=4`):

$$I_{t,s} = \sum_j w_{t,j} \, \mathrm{ReLU}(q_{t,j} \cdot k_s)$$

Variant 6: Gated Attention

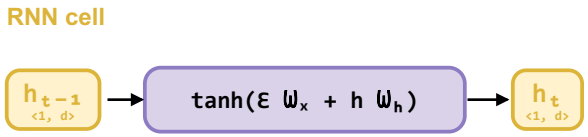
Qwen3-Next, Qwen3.5, Arcee Trinity Large 400B (typically combined with Hybrid Attention)

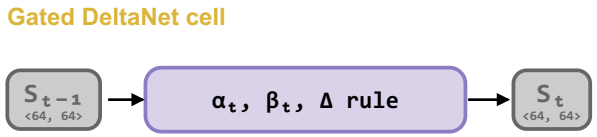


Essentially we just **add an extra weight matrix** (a gate) before adding the residual. The original Gated Attention paper found it adds some non-linearity and gets rid of the "attention sink", and Qwen3-Next uses it for the **full-attention layers** it keeps inside a hybrid stack.

Variant 7: Hybrid Attention (Gated DeltaNet)

Qwen3-Next (very tricky)





Like an RNN

- **Fixed-size state** carried token to token
- **Constant memory and work per token** at decode: no growing KV cache
- **Gates** decide what to forget (α) and what to write (β), like LSTM / GRU
- Qwen3-Next: 3 of every 4 layers are Gated DeltaNet

Not like an RNN

- State is a **matrix of key → value pairs** (64 × 64 per head here), not a vector
- Update is **linear in S** (no tanh): chunk-parallel training, no BPTT
- Read-out q_t S_t is linear attention (no softmax): weaker exact recall, so 1 Gated Attention layer per 3

Variant 7: The Delta Rule

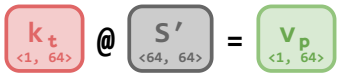
1 Decay

forget a little



2 Recall

what does S' hold for key k_t?



3 Error

true value minus recall

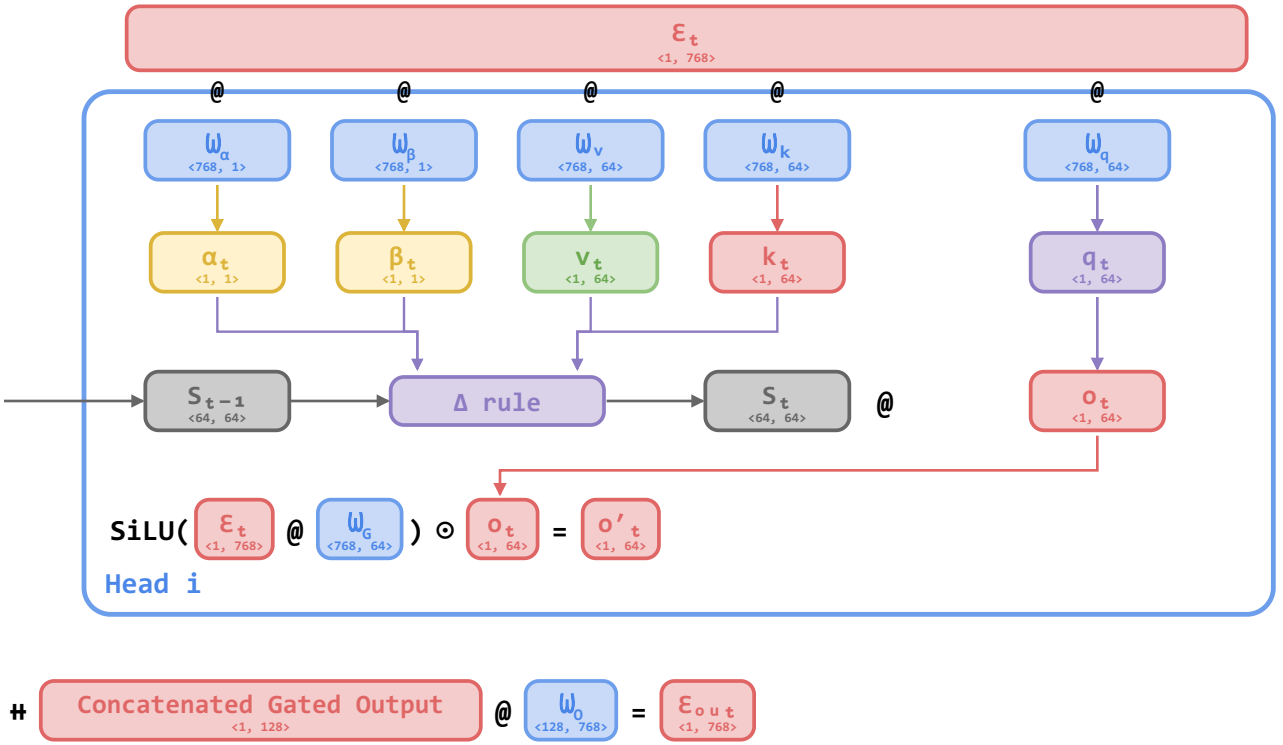


4 Write

correct S' by β_t × error



Variant 7: Gated DeltaNet



Flash Attention (very important)

- **Same answer**, computed smarter: Flash Attention doesn't change what attention does
- A GPU has two kinds of memory: a **small, very fast workbench** and a **huge, slow warehouse**
- Standard attention keeps making **trips to the warehouse**: it stores the giant N × N score table, then fetches it back
- The GPU spends most of its time **waiting on those trips**, not doing math
- Flash works on **small tiles at the workbench** and only writes down the final result
- Payoff: **far less memory** and several × faster, so much longer inputs fit
- The trick: softmax can be updated tile by tile using **running totals**
- Works alongside the variants in this lecture: they change **what** is computed, Flash changes **how**

Technically, the workbench is SRAM and the warehouse is HBM